The Commit

GitHub edition
No. 005 · Vol. 1
Monday, 9 November 2026

15 trending repos **2** deep dives · p.2

			STARS	GAINED
01	vercel-labs/**agent-desktop** Gives AI agents desktop controls through accessibility trees, letting them inspect interfaces and act on named elements.	Rust	20.3k	+13.7k
02	NVIDIA/**auto-ontology** Builds shared definitions over business data so AI agents can query databases with more traceable answers.	Python	10.9k	+9.3k
03	Donchitos/**Claude-Code-Game-Studios** Helps Claude Code turn game ideas into playable browser prototypes, then guides revisions through documented workflows.	Shell	37.4k	+5.2k
04	meta/**astryx** Provides customisable React interface components and themes for designers, developers and AI assistants building applications together.	TypeScript	19.3k	+4.4k
05	vercel-labs/**agent-browser** Lets AI agents inspect web pages as accessibility trees and operate browsers through individual shell commands.	Rust	48.3k	+2.2k
06	zed-industries/**zed** Offers developers a collaborative code editor with desktop downloads for macOS, Linux and Windows operating systems.	Rust	94.6k	+2.2k
07	anomalyco/**hex** Transcribes speech locally and pastes it into the current app, helping people dictate across desktop software.	Rust	6k	+2.1k
08	longbridge/**gpui-kit** Supplies Rust components and application infrastructure for developers building desktop interfaces with the GPUI rendering framework.	Rust	20.1k	+1.2k
09	liguobao/**ds-harness-remote** Connects phones, computers and browsers to coding workspaces running on a user's existing remote work computer.	TypeScript	2.4k	+1.1k
10	elie222/**rakazo** Runs persistent AI teammates with memory, scheduled routines and computer access using models selected by users.	TypeScript	7.7k	+1k
11	astral-sh/**ruff** Checks and formats Python code, bringing automated fixes and many common development checks into one tool.	Rust	51.1k	+881
12	can1357/**oh-my-pi** Combines an AI coding agent with integrated development tools for programmers working across multiple model providers.	TypeScript	38.4k	+870
13	superplanehq/**superplane** Coordinates coding agents, checks and approvals to turn suitable engineering backlog issues into changes for review.	Go	9.4k	+855
14	multica-ai/**multica** Gives teams a shared workspace to assign issues to coding agents, track progress and review results.	Go	55.4k	+841
15	prometheus/**prometheus** Collects service metrics over time and evaluates rules, helping operators graph system behaviour and trigger alerts.	Go	67.6k	+832

```
auto-ontology — Python — +9.3k ★
~ % # It makes human certification of business definitions part of the path from a question to an answer.
~ % gh repo clone NVIDIA/auto-ontology
```

Giving business data a shared meaning

Nvidia's library builds a reviewable definition layer between an AI agent and the databases it queries.

A database can hold the right numbers without explaining what they mean. NVIDIA Auto Ontology builds a shared context layer from enterprise metadata. Its README describes helping agents find data, reconcile definitions across systems and return answers with provenance. The aim is to make business analytics easier to inspect, with definitions that people can review.

The workflow has three stages: generate, certify and answer. First, it reads schemas, profiles columns and samples values, then proposes business terms, table relationships and reusable SQL expressions. Definitions are stored in Postgres, with embeddings for retrieval. People test the draft against real questions, refine definitions and certify them. For each question, the agent retrieves candidate definitions, constructs SQL, checks filter values and repairs failed queries. The answer includes its SQL and the steps taken. A documented example asks about advertising spend absent from the sample database: the agent reports the gap and seeks clarification.

A web interface supports chat and catalog inspection. Agents can connect through MCP, a command line or an API; the documented local setup uses Docker Compose and NVIDIA inference endpoints. These mechanisms explain the project's appeal, but do not establish that its generated definitions or answers are always correct. Human review remains part of the documented process.

Source: project README

```
prometheus — Go — +832 ★
~ % # Its pull model and autonomous servers offer a clear explanation of how software systems report their health.
~ % gh repo clone prometheus/prometheus
```

A monitoring server that asks for numbers

Prometheus gathers measurements on a schedule, then turns them into queries, graphs and alerts for service operators.

Prometheus records how measurements change over time. Its README describes a monitoring loop: collect metrics from configured targets at regular intervals, evaluate rules, display results and trigger alerts when specified conditions occur. The Cloud Native Computing Foundation project gives operators a way to inspect systems and services through the measurements those systems expose.

Its data model assigns each time series a metric name and key-value dimensions. PromQL, its query language, works with those attributes. Targets can be listed in configuration or found through service discovery. Prometheus normally collects measurements by making HTTP requests to targets; batch jobs can instead send them through an intermediary gateway. Individual server nodes are autonomous, without depending on distributed storage. The project also supports horizontal and hierarchical federation, with several ways to graph data and build dashboards. These are architectural choices, rather than guarantees that every deployment will meet a particular performance target.

The README recommends the latest production binary and also supplies container images and source-build instructions. It draws a boundary around reuse: the main repository builds a standalone program, rather than a general Go library. Maintainers warn that internal interfaces can change and library users may encounter errors. Running the monitoring service and building software around its internals therefore involve different expectations.

Source: project README